

Models and Tools for Cyber-Physical Systems
Exercise sheet 11
WITH SOLUTIONS

Exercise 1: Parking a Car

Consider the car model from the course

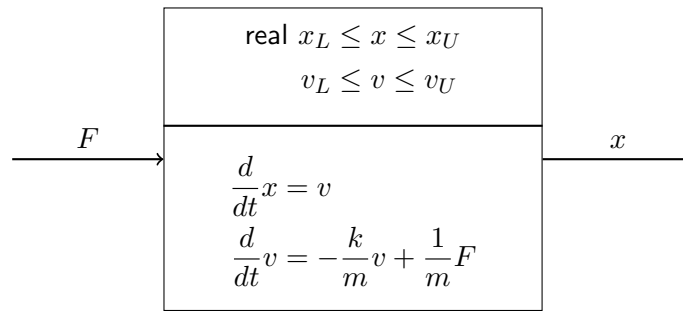
$$\frac{d}{dt}x(t) = v(t) \qquad \frac{d}{dt}v(t) = -\frac{k}{m}v(t) + \frac{1}{m}F(t),$$

where x is the position, v the velocity and F the force applied in case of mass m and friction coefficient k . Starting at position $x(0) = -100$, we wish to bring the car to the point 0. To this end, we want to construct a proportional controller K_p for $m = 1000$ (kg) and $k = 50$.

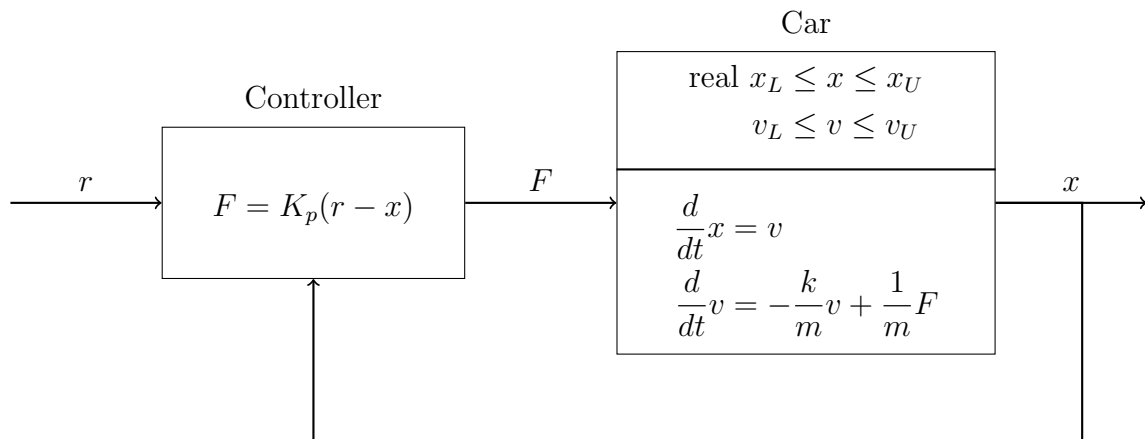
- (a) Assuming that we have no feedforward control and measure only the position of the car, provide a continuous-time component for the parking model in block diagram form.
- (b) Create a continuous-time component of the proportional controller in block diagram form and indicate how the controller component is connected to the car model component. Finally, combine them together into a single continuous-time component.
- (c) Using your numerical solver from last sheet, tune your controller, i.e., find appropriate values for K_p , such that the car is parked at $x = 0$ within $t = 100$ (s). Describe the quality of your controller in terms of steady state error, settling time, overshoot and rise time.
- (d) Derive a formula for the Proportional-Derivative (PD) controller that brings the car to the state $(x, v) = (100, 0)$. Here a PD controller is a PID controller whose integral component has coefficient zero, i.e., $K_i = 0$. Moreover, implement the PD car controller using your numerical solver and tune the values of K_p and K_d . Describe the quality of your controller in terms of steady state error, settling time, overshoot and rise time.

.....Solution sketch

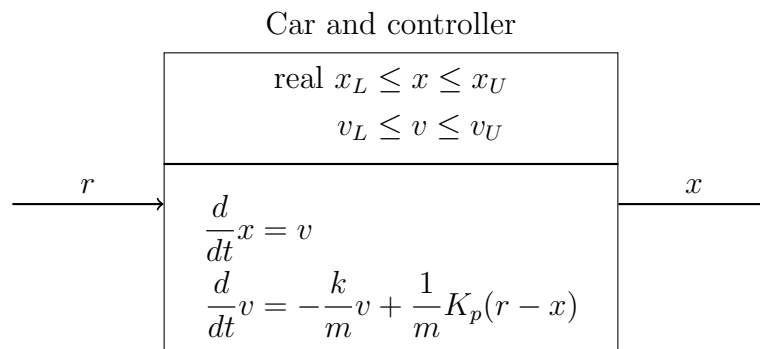
- (a) The model is already given as a set of first-order differential equations, so we do not have to do any rewriting. The continuous-time component of the car is shown below.



- (b) The continuous-time component of the controller is shown below. Note that a proportional controller does not have an internal state.



The combined continuous-time component of the car and the controller is shown below.



- (c) You can use the solution code from last sheet where you define function `f` to be as follows.

```

1  def f(x, t):
2      m = 1000
3      k = 50
4      Kp = 10
5      r = 0
6      der_x = x[1]
7      der_v = -k / m * x[1] + Kp * (r - x[0]) / m
8      return [der_x, der_v]

```

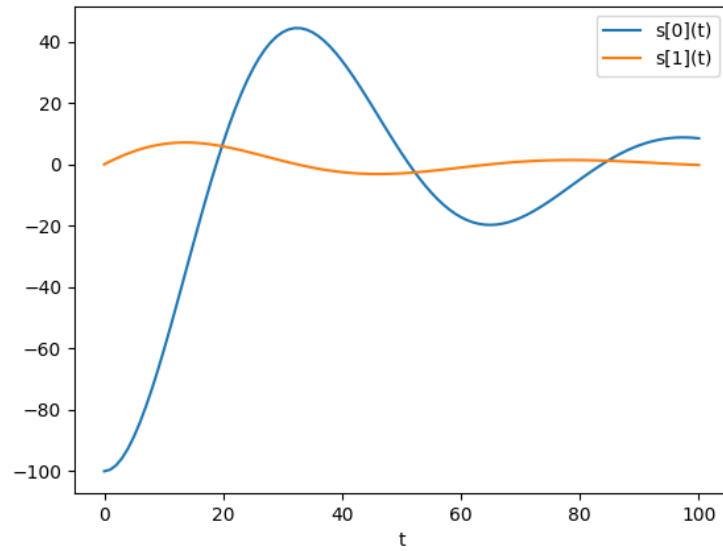


Figure 1: Car model state trajectories with only proportional controller $K_p = 10$.

Figure 1 shows a plot of position x (blue) and velocity v (orange) for a gain of $K_p = 10$.

By changing the value of K_p you can observe that low values of K_p result in a controller that has no overshoot but a very long settling time, while high values of K_p result in a fast rise time but significant overshoot and not really a fast settling time. So a proportional controller does not really have the desired performance.

- (d) We first note that the error is $e = (r - x) = (100 - x)$. Hence, $u_p = K_p(100 - x)$ and $u_d = K_d \frac{d}{dt}(100 - x) = -K_d v$. With this, the overall control is thus $F = u = u_p + u_d = K_p(100 - x) - K_d v$.¹

A possible Python code for the PD controller is given below. The resulting trajectories of the car's state variables are shown in Figure 2 for $K_p = 20$ and $K_d = 200$ (the values in the script below).

```

1 def f(x, t):
2     m = 1000
3     k = 50
4     Kp = 20
5     Kd = 200
6     r = 0
7     F = Kp * (r - x[0]) - Kd * x[1]
8     der_x = x[1]
9     der_v = -k / m * x[1] + F / m
10    return [der_x, der_v]

```

¹Note that in this example the derivative of the error can be expressed directly using state variables of the system. In general, it would be possible to implement a PD controller by approximating $\frac{d}{dt}e$ via the differential quotient. To this end, one would need to know the values of t and e of the foregoing step.

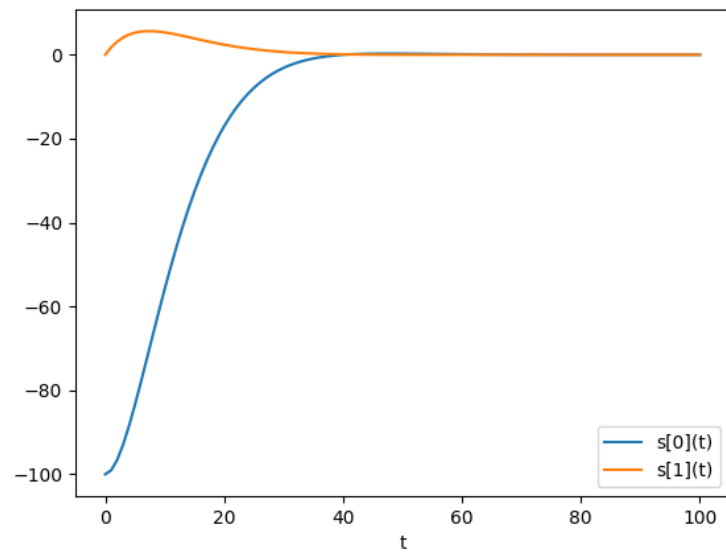


Figure 2: Car model state trajectories with a PD controller using $K_p = 20$ and $K_d = 200$.